

Programação Orientada a Objetos

Prof. Dr. Josenalde Barbosa de Oliveira

josenalde.oliveira@ufrn.br

<https://github.com/josenalde/poo>

Tratamento de erros (exceções)

- Uso do bloco try...catch...finally para capturar vários tipos de Erros

```
public class Principal {  
    public static void main(String[] args) {  
        int x = 3;  
        int y = 0;  
        double res = Double.NaN;  
        try {  
            res = x / y; // bloco com lógica normal, funcionalidades  
        } catch (ArithmeticException e) {  
            // se usar Exception é a superclasse  
            res = Double.NaN; // pode exibir o erro no objeto e  
            System.out.println(e);  
        } finally { // executado de qualquer forma, com ou sem erro  
            System.out.println("Resultado: " + res);  
        }  
    }  
}
```

Tratamento de erros (exceções)

- Uso do bloco try...catch...finally para capturar vários tipos de Erros

```
public class Principal {  
    public static void main(String[] args) {  
        Pessoa pessoa = null;  
        try {  
            pessoa.setNome("John");  
        } catch (NullPointerException e) {  
            pessoa = new Pessoa();  
        } finally { // executado de qualquer forma, com ou sem erro  
            pessoa.setNome("John");  
        }  
    }  
}
```

Types of Exceptions

```
graph TD; A[Types of Exceptions] --> B[Use-Defined Exception]; A --> C[Built-in Exception]; C --> D[Checked Exceptions]; C --> E[Unchecked Exceptions]; D --> D1[• ClassNotFoundException]; D --> D2[• InterruptedException]; D --> D3[• IOException]; D --> D4[• InstantiationException]; D --> D5[• SQLException]; D --> D6[• FileNotFoundException]; E --> E1[• ArithmeticException]; E --> E2[• ClassCastException]; E --> E3[• NullPointerException]; E --> E4[• ArrayIndexOutOfBoundsException]; E --> E5[• ArrayStoreException]; E --> E6[• IllegalStateException];
```

Use- Defined Exception

Built-in Exception

Checked Exceptions

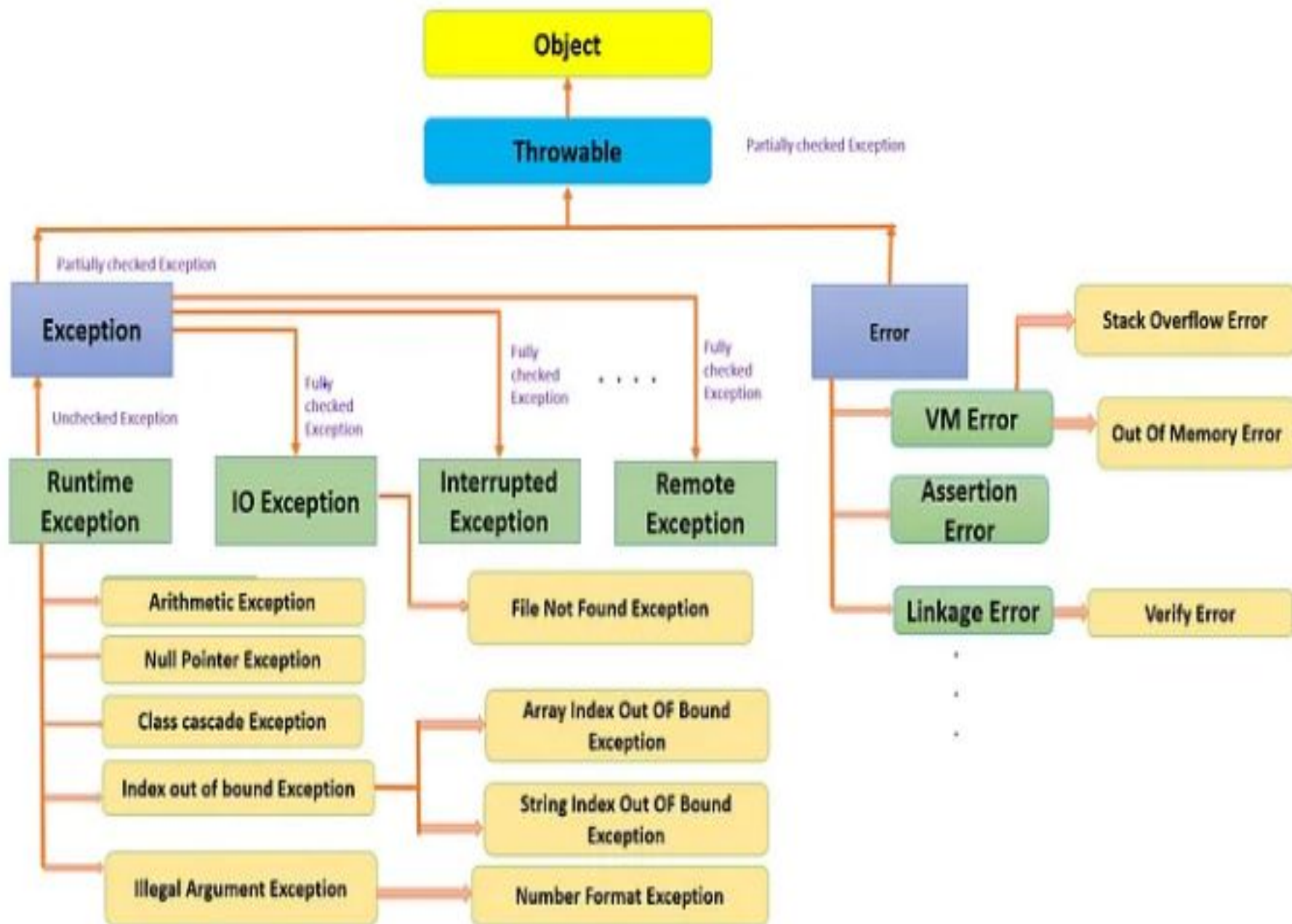
- ClassNotFoundException
- InterruptedException
- IOException
- InstantiationException
- SQLException
- FileNotFoundException

Unchecked Exceptions

- ArithmeticException
- ClassCastException
- NullPointerException
- ArrayIndexOutOfBoundsException
- ArrayStoreException
- IllegalStateException

Em Java, exceções verificadas (checked exceptions) são erros previsíveis e recuperáveis que o compilador exige que sejam tratados ou declarados, como a falta de um arquivo (`FileNotFoundException`).

Já as exceções não verificadas (unchecked exceptions) são erros de programação que ocorrem em tempo de execução, como divisão por zero (`ArithmeticException`), e não são obrigadas a serem tratadas, embora possam ser.



Tratamento de erros

- Podemos criar quantos blocos **catch** achemos necessários para tratar exceções que podem ocorrer dentro do **try**, ao fim, ainda coloca-se um **catch** geral, com Exception e. [Não coloque Exception antes das exceções específicas, senão só capta a Exception geral, da superclasse]
- **finally** é importante para liberar recursos, fechar conexões com banco de dados, liberar tokens de segurança, ou simplesmente, executar instruções independente de ocorrer ou não exceção
- O **finally** só não é executado se dentro do bloco **try** há um método **exit()**. Neste caso os recursos são liberados diretamente pelo sistema operacional.
- Pode-se deslocar o tratamento para outro local, não necessariamente onde ocorre a exceção. Exemplo:

Tratamento de erros - no próprio método

```
public class Robo {  
    public void calcularVelocidade(int tempo, int distancia) {  
        try {  
            this.velocidade = distancia / tempo;  
        } catch (ArithmeticException e) {  
            //notificar velocidade inválida  
        }  
    }  
}
```

Tratamento de erros - lançando para outro local!!

```
public class Robo {  
    public void calcularVelocidade(int tempo, int distancia) throws  
    ArithmeticException {  
        this.velocidade = distancia / tempo;  
    }  
}
```


Tratamento de erros - recebendo na Principal

```
public class Principal {  
    public static void main(String[] args) {  
        Robo r1 = new Robo(); //assuma velocidade=0 aqui  
        try {  
            //ler tempo, ler dist  
            r1.calcularVelocidade(tempo, dist);  
        } catch (ArithmeticException e) {  
            // informar erro  
        } finally {  
            System.out.println(r1.getVelocidade());  
        }  
    }  
}
```

Tratamento de erros

```
public class Jogador {  
    public void setIdade(int idade) throws Exception {  
        Exception e = new Exception("Jogador menor de 18 anos");  
        if (idade < 18) throw e;  
        this.idade = idade;  
    }  
}
```

```
public class Principal {  
    public static void main(String[] args) {  
        Jogador j = new Jogador();  
        try {  
            j.setIdade(7); //se lançar, é capturado abaixo  
        } catch (Exception ex) {  
            // informar erro  
        }  
    }  
}
```

Tratamento de erros – classe de exceções

```
public class IdadeJogadorException extends RuntimeException {  
    public IdadeJogadorException(String message) {  
        super(message);  
    }  
}
```

```
public class Jogador {  
    public void setIdade(int idade) throws IdadeJogadorException {  
        if (idade < 18) {  
            throw new IdadeJogadorException("Idade menor que 18 anos");  
        }  
        this.idade = idade;  
    }  
}
```

Tratamento de erros – classe de exceções

```
public class Principal {  
    public static void main(String[] args) {  
        Jogador j = new Jogador();  
        try {  
            j.setIdade(7);  
        } catch (IdadeJogadorException ex) {  
            // informar erro ex  
        }  
    }  
}
```

Enumerações: lista de constantes

```
public class Main {  
    enum Level {  
        LOW,  
        MEDIUM,  
        HIGH  
    }  
  
    public static void main(String[] args) {  
        Level myVar = Level.MEDIUM;  
        System.out.println(myVar); //MEDIUM  
        System.out.println(Level.valueOf("MEDIUM").ordinal()); //1  
        System.out.println(myVar==Level.LOW); //false  
    }  
}
```

Ver outros usos aqui: https://www.w3schools.com/java/java_enums.asp